used in residualization.

In [*]:
```
# -------------------- 7. Momentum test battery --------------------
# tests run on the DISCOVERED CANDIDATES (target universe) = the product claim
# uses scores (cosine sim per theme) + tgt_returns + ff_factors (needs MOM column)

from src.momentum_test import run_momentum_battery, summarize_battery

# ensure the momentum factor is available for the spanning test
# ensure the momentum factor is available for the spanning test
if 'MOM' not in ff_factors.columns:
    print('WARNING: MOM missing from ff_factors -- re-pulling momentum factor directly...')
    from src.residualize import get_ff_factors
    ff_refetch = get_ff_factors(start=str(tgt_returns.index[0].date()),
                                end=str(tgt_returns.index[-1].date()), freq='weekly')
    if 'MOM' in ff_refetch.columns:
        ff_factors = ff_factors.join(ff_refetch[['MOM']], how='left')
        print('  MOM recovered via re-pull.')
    else:
        print('  MOM still unavailable -- spanning test (Test C) skipped this run.')


mom_results = run_momentum_battery(
    scores, tgt_returns, ff_factors,
    horizon_weeks=4, verbose=True
)

print('\n' + '='*60)
print('CROSS-THEME SUMMARY (candidate-level)')
print('='*60)
mom_summary = summarize_battery(mom_results)
mom_summary
```

How to read the summary:

- `A_theme_spread_in_mom` > 0 : theme still sorts returns *within* momentum buckets (theme ≠ momentum)
- `B_theme_coef_joint` significant (|t| > ~2) : theme predicts returns *after* controlling for momentum
- `C_alpha_ann` significant : theme long-short portfolio is not spanned by momentum
- Compare `B_theme_coef_alone` vs `B_theme_coef_joint` : if the coef barely shrinks when momentum is added, theme is largely independent of momentum

In [27]:
```
# -------------------- double-sort heatmaps for a few themes --------------------
import matplotlib.pyplot as plt
import seaborn as sns

# show the double-sort grid for the 3 themes that own clean factors
show_themes = [t for t in ['AI Infrastructure', 'Defense', 'Cybersecurity'] if t in mom_results]
if show_themes:
    fig, axes = plt.subplots(1, len(show_themes), figsize=(5*len(show_themes), 4))
    if len(show_themes) == 1: axes = [axes]
    for ax, theme in zip(axes, show_themes):
        grid = mom_results[theme]['double_sort']['grid'] * 100
        sns.heatmap(grid, annot=True, fmt='.1f', cmap='RdYlGn', center=0, ax=ax,
                    cbar_kws={'label': 'fwd return %'})
        ax.set_title(f'{theme}\n(rows=momentum Q, cols=theme Q)')
        ax.set_xlabel('theme score quintile')
        ax.set_ylabel('momentum quintile')
    plt.tight_layout()
    plt.savefig('outputs/fig_momentum_double_sort.png', dpi=150)
    plt.show()
    print('reading: if color brightens LEFT->RIGHT within each row,')
    print('theme predicts returns independent of momentum (the row).')
```